Feb 20th HW 18

Part 1 – Core Concepts

- Topic: A logical category/channel where messages are stored. Producers publish messages to topics and consumers read from them.
- Partition: Each topic is divided into partitions. A partition is an ordered, immutable log of messages identified by offsets. Partitions enable parallelism.
- Broker: A Kafka server instance. Brokers store partitions and handle read/write requests. Multiple brokers form a Kafka cluster.
- Producer: Application that sends messages to Kafka topics. Can control partitioning and delivery guarantees.
- Consumer: Application that pulls messages from Kafka topics.
- Consumer Group: A group of consumers sharing the same group ID. Each partition is consumed by only one consumer within the same group.
- Offset: A unique identifier for each message within a partition. Kafka tracks offsets per consumer group.
- Zookeeper: Used (in older versions) for broker coordination, leader election, and metadata management.

Part 2 – Concept Questions

1. N (partitions) and M (consumers): If $N \geq M$, each consumer gets at least one partition. If $N < M$, extra consumers remain idle since one partition can only be consumed by one consumer in the same group.
2. How brokers work with topics: Topics are divided into partitions, and partitions are distributed across brokers. Each partition has a leader and replicas for fault tolerance.
3. Push or Pull? Kafka uses a pull model. Consumers actively pull messages from brokers.
4. Avoid duplicate consumption: Use manual offset commits after processing, enable idempotent producer, or implement exactly-once semantics.
5. If some consumers are down: Kafka triggers rebalance and reassigns partitions. No data loss occurs because offsets and logs are stored safely.
6. If entire consumer group is down: No data loss occurs. Messages remain in Kafka until retention expires.
7. Consumer lag: Lag = Latest Offset – Committed Offset. Resolve by increasing partitions, consumers, or optimizing processing.
8. Message delivery tracking: Kafka stores offsets in the internal topic '__consumer_offsets'.

9. Kafka vs RabbitMQ vs MySQL: Kafka is a distributed log optimized for high throughput streaming. RabbitMQ is a traditional message queue. MySQL is a database and not designed for streaming workloads.